

SLIDE 1 — What is Working with a 2D Array?

Definition:

Working with a **2D array** means storing, accessing, and processing data using **rows and columns**.

Simple idea

Create → **Store** → **Access** → **Process**

Example:

```
int matrix[2][3] = {  
    {10, 20, 30},  
    {40, 50, 60}  
};
```

	0	1	2	
0	10	20	30	
1	40	50	60	

 **Remember: 2D Array = Rows + Columns**

SLIDE 2 — Taking Input in a 2D Array

We can use **nested loops** to fill a 2D array.

Example

```
int matrix[2][3];  
  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 3; j++) {  
        scanf("%d", &matrix[i][j]);  
    }  
}
```

```
}
```

Explanation:

i → Row

j → Column

The loop fills:

matrix[0][0] → matrix[0][1] → matrix[0][2]

matrix[1][0] → matrix[1][1] → matrix[1][2]

★ Remember

● Outer loop → **Rows**

● Inner loop → **Columns**

SLIDE 3 — Displaying a 2D Array

Nested loops can also display all elements.

Example

```
int matrix[2][3] = {  
    {10, 20, 30},  
    {40, 50, 60}  
};  
  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 3; j++) {  
        printf("%d ", matrix[i][j]);  
    }  
    printf("\n");  
}
```

Output:

```
10 20 30  
40 50 60
```

Explanation:

The outer loop moves **row by row**, while the inner loop moves through the **columns**.

🧠 Remember: `matrix[i][j]` → Current element

SLIDE 4 — Basic Operations on a 2D Array

We can perform calculations on all elements.

Example: Find Sum

```
int sum = 0;
```

```
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 3; j++) {  
        sum += matrix[i][j];  
    }  
}
```

For:

```
10 20 30  
40 50 60
```

Calculation:

```
10 + 20 + 30 + 40 + 50 + 60  
    ↓  
   210
```

Output:

Sum = 210

🧠 Basic Pattern

ROW → COLUMN → ACCESS → PROCESS

SLIDE 5 — RECAP

Working with 2D Arrays!

Flow

CREATE



INPUT



ACCESS



PROCESS



OUTPUT

5 Things to Remember

1. 2D arrays store data in rows and columns
2. Two indexes are used: `[row][column]`
3. Nested loops are commonly used
4. Outer loop handles rows
5. Inner loop handles columns

Final Example

```
int matrix[2][2] = {
    {10, 20},
    {30, 40}
};

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
```

Output:

```
10 20
30 40
```

Memory Trick

i → ROW ↑

j → COLUMN ↔

matrix[i][j] → ELEMENT 🎯

QUESTIONS

1. What is commonly used to process a 2D array?

- A) Single variable
- B) Nested loops
- C) Only **if**
- D) Only **printf()**

💡 **Hint:** One loop handles rows and another handles columns.

📖 **Glossary:** Nested loop → A loop inside another loop.

✅ **Answer: B**

2. In **matrix[i][j]**, what does **i** represent?

- A) Column
- B) Row
- C) Value
- D) Size

💡 **Hint:** The first index represents the row.

📖 **Glossary:** Row → Horizontal group of elements.

✅ **Answer: B) Row**

3. What is the output?

```
int matrix[2][2] = {  
    {10, 20},
```

```
{30, 40}  
};
```

```
printf("%d", matrix[1][0]);
```

 **Hint:** Go to row 1, column 0.

 **Glossary: Access** → Retrieve a particular element.

 **Answer:** 30

4. Fill in the blanks

Complete the nested loops:

```
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 3; j++) {  
        printf("%d ", matrix[____][____]);  
    }  
}
```

 **Hint:** Use the row variable first and column variable second.

 **Glossary: Column** → Vertical group of elements.

 **Answer:** `[i][j]`

5. How many elements are in `int matrix[3][4]`?

- A) 7
- B) 12
- C) 3
- D) 4

 **Hint:** Rows × Columns.

 **Glossary: Element** → A single value stored in an array.

 **Answer:** B) 12

6. Which loop usually handles the columns?

- A) Outer loop
- B) Inner loop
- C) Both always
- D) No loop

 **Hint:** The inner loop changes faster.

 **Glossary:** Inner loop → The loop placed inside another loop.

 **Answer:** B) Inner loop

7. Arrange the steps in the correct order:

- A. Process the element
- B. Create the 2D array
- C. Access using `[i][j]`
- D. Use nested loops

 **Hint:** Create → Loop → Access → Process.

 **Glossary:** Process → Perform an operation on data.

 **Answer:** B → D → C → A